INDORE INSTITUTE OF SCIENCE AND TECHNOLOGY

SKILL UP – 26 batch

Mrs. Varsha Zokarkar

NUMBERS CODING & ADVANCED IN NUMBERS

1. SERIES PROBLEM SOLVING
2. LEAP YEAR
3. DIGIT SUM , REVERSE & PALINDROME OF A NUMBER
4. ARMSTRONG NUMBER
5. PRIME NUMBER
6. PRIME FACTORS OF A NUMBER
7. HCF/GCD OF TWO NUMBERS
8. LCM
9. BINARY TO DECIMAL CONVERSION
10. OCTAL TO DECIMAL CONVERSION
11. HEXADECIMAL TO DECIMAL CONVERSION

ARITHMETIC PROGRESSION

General Form:
An AP can be represented as a, a + d, a + 2d, a + 3d, ... where 'a' is the first term.
nth term:
The nth term of an AP can be found using the formula: an = a + (n-1)d, where an is the nth term, a is the first term, n is the number of terms, and d is the common difference.
Sum of n terms:
The sum of the first n terms of an AP can be calculated using the formula: Sn = n/2 [2a + (n - 1)d]
or
Sn = n/2 [a1 + an], where Sn is the sum of the first n terms, a is the first term, n is the number of terms, d is the common difference, and an is the nth term.

EXAMPLE -

- Sum of First N Natural numbers
- Sum of numbers in a given range
- Leetcode 1502. can make arithmetic progression from sequence

GEOMETRIC PROGRESSION

A geometric progression can be represented as: a, ar, ar^2, ar^3, ... where 'a' is the first term and 'r' is the common ratio.

- Formulas:
- **nth term:** $a_n = ar^{n-1}$
- **Sum of first n terms (S_n):**
  - If $r \neq 1$, $S_n = a(1 - r^n) / (1 - r)$
  - If $r > 1$, $S_n = a(r^n - 1) / (r - 1)$
- **Sum of an infinite geometric progression (if $|r| < 1$):** $S\infty = a / (1 - r)$

LEAP YEAR

- Divide the year by 4: If the remainder is not 0, it's not a leap year.
- If divisible by 4:
  - If it's an end-of-century year (e.g., 1900, 2000), then divide by 400. If the remainder is 0, it's a leap year; otherwise, it's not.
  - If it's not an end-of-century year, it's a leap

Leetcode 1154: Day of the year

REVERSE INTEGER (LEETCODE 7: REVERSE INTEGER)

- Create a variable ans and initialize it to 0. This variable will hold the reversed number.
- Processing Each Digit:
- Use a while loop to iterate as long as x is not zero. In each iteration:
- Extract the last digit of x using x % 10 and store it in the variable digit.
- Check if appending the digit to ans would cause overflow or underflow. This is done by comparing ans with INT_MAX / 10 and INT_MIN / 10.
- If ans is greater than INT_MAX / 10, multiplying it by 10 and adding any digit would exceed INT_MAX.
- Similarly, if ans is less than INT_MIN / 10, multiplying it by 10 and adding any digit would go below INT_MIN.
- If the overflow or underflow condition is detected, return 0.
- Update ans by multiplying it by 10 and adding the digit.
- Remove the last digit from x by performing integer division x / 10.
- Return Result:
- After the loop ends, ans contains the reversed integer, and it is returned as the result.

PALINDROME NUMBER- LEETCODE 9: PALINDROME NUMBER

- If *x* is negative, it's not a palindrome (because of the - sign).
- Store the original number in a variable *y*.
- Reverse the digits of *x*:
- Use a *while* loop to extract digits from *x* (using *x* % 10) and build the reversed number *rev*.
- Update *rev* = *rev*∗10 + *x* % 10 and reduce *x* by *x*/=10.
- After the loop, compare the reversed number *rev* with the original number *y*.
- If they are equal, return *true*.
- Else, return *false*.

•The sum of the cubes of its digits is 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.
•Since the sum of the cubes (153) is equal to the original number (153), it's an Armstrong number.

ARMSTRONG NUMBER

An Armstrong number is a number that is equal to the sum of its own digits each raised to the power of the number of digits in the number. For example, 153 is an Armstrong number because 1^3 + 5^3 + 3^3 = 153.

Example:

Let's consider the number 153.

It has three digits (a three-digit number).

The sum of the cubes of its digits is 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153.

Since the sum of the cubes (153) is equal to the original number (153), it's an Armstrong number.

GREATEST COMMON DIVIDER (GCD) / HIGHEST COMMON FACTOR (HCF)

Euclidean algorithm -

```
int gcd (int a, int b)
{
    if (b == 0) return a;
    else return gcd (b, a % b);
}
```

Leetcode 2447 : numbers of subarrays GCD equal to k

LOWEST COMMON MULTIPLE (LCM)

The key things to remember are:

- $LCM(a,b)=(a*b)/GCD(a,b)$
- $LCM(a,b,c)=LCM(LCM(a,b),c)$
- $LCM(a,b)=l>k$ implies that $LCM(l,c)>k$

LeetCode 2447 : Number of subarrays with LCM equal to k.

PRIME NUMBER

Prime Numbers	Composite Numbers
All Prime Numbers have only two factors such as 1 and the number itself. For example, 5 has only two factors 1 and 5.	All composite numbers must have any factor other than 1 and the number itself. For example, 6 has 2 and 3 as its factors other than 1 and 6.
All Primes are Odd numbers except 2.	Composites can be either odd or even.
Some of the first few primes are 2, 3, 5, 7, 11, 13, 17, etc.	Some of the first few composites are 4, 6, 8, 9, 10, 12, 14, 15, etc.

Leetcode 3115 : maximum prime difference

PRIME NUMBER

```
bool isPrime(int n) {
    // Check if n is 1 or 0
    if (n <= 1)
        return false;
    // Check if n is 2 or 3
    if (n == 2 || n == 3)
        return true;
    // Check whether n is divisible by 2 or 3
    if (n % 2 == 0 || n % 3 == 0)
        return false;
    // Check from 5 to square root of n
    // Iterate i by (i+6)
    for (int i = 5; i <= sqrt(n); i = i + 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return false;

    return true;
}
```

PRIME NUMBER IN GIVEN RANGE - SIEVE OF ERATOSTHENES

The Sieve of Eratosthenes efficiently finds all primes up to n by repeatedly marking multiples of each prime as non-prime, starting from 2. This avoids redundant checks and quickly filters out all composite numbers.

Step By Step Implementations:

1. Initialize a Boolean array prime[0..n] and set all entries to true, except for 0 and 1 (which are not primes).
2. Start from 2, the smallest prime number.
3. For each number p from 2 up to √n:
4. If p is marked as prime(true):
5. Mark all multiples of p as not prime(false), starting from p * p (since smaller multiples have already been marked by smaller primes).
6. After the loop ends, all the remaining true entries in prime represent prime numbers.

SIEVE OF ERATOSTHENES (LEETCODE 204:COUNT PRIMES)

```
vector<int> sieve(int n) {

    // creation of boolean array
    vector<bool> prime(n + 1, true);
    for (int p = 2; p * p <= n; p++) {
        if (prime[p] == true) {

            // marking as false
            for (int i = p * p; i <= n; i += p)
                prime[i] = false;
        }
    }

    vector<int> res;
    for (int p = 2; p <= n; p++){
        if (prime[p]){
            res.push_back(p);
        }
    }
    return res;
}
```

•Prime factors of 45 are 3 and 5 because 3 x 3 x 5 = 45.
•Prime factors of 60 are 2, 3, and 5 because 2 x 2 x 3 x 5 = 60.

PRIME FACTORS

Prime factors are prime numbers that, when multiplied together, result in a given number. For instance, the prime factors of 20 are 2 and 5, since 2 x 2 x 5 = 20.

Examples:

Prime factors of 12 are 2 and 3 because 2 x 2 x 3 = 12.

Prime factors of 30 are 2, 3, and 5 because 2 x 3 x 5 = 30.

Prime factors of 45 are 3 and 5 because 3 x 3 x 5 = 45.

Prime factors of 60 are 2, 3, and 5 because 2 x 2 x 3 x 5 = 60.

Leetcode 2521 : Distinct prime factors of product of array.

BINARY TO DECIMAL CONVERSION

```
int binaryToDecimal(string n)
{
    string num = n;
    int dec_value = 0;
    // Initializing base value to 1, i.e 2^0
    int base = 1;
    int len = num.length();
    for (int i = len - 1; i >= 0; i--) {
        if (num[i] == '1')
            dec_value += base;
        base = base * 2;
    }

    return dec_value;
}
```

Leetcode 191: Number of 1 bits

DECIMAL TO HEXADECIMAL CONVERSION (LEETCODE 405: CONVERT A NUMBER TO HEXADECIMAL.)

- We first handle the edge case i.e. when num==0.
- We initialize a string s which will store the hexadecimal characters. Along with it, we also initialize a character c and integer n.
- We will use a loop which will run eight times.

  a. Now we will consider the binary representation of num for the conversion. When we do this process in real life, we group the binary digits in a group of 4 from right to left and convert each group to the hexadecimal number it represents. We are going to use the same logic here too.

  b. Now, we will apply & operation on num and 15 (binary=1111) and store the result in n. Using an if-else clause, we will convert the integer n to the corresponding character using ASCII values and typecasting and store the result in c.
- c. We will append the character c to the front of the string and right shift the bits of num 4 times.
- We will use a while loop to get rid of the trailing '0' in s.
- Return the string s.